PsyberShield User Guide

PsyberShield is a Python-based security visibility and response tool for small servers and web applications.
This guide is a living document. It describes the current CLI behavior, the command set, the important flags, and what to expect from each workflow.

The preferred CLI command is ‘pshield‘. ‘psybershield‘ and ‘turan‘ remain compatibility aliases during the transition.

For version history, see docs/changelog.md.
This guide and the changelog should be updated together when PsyberShield’s user-facing behavior changes.

What PsyberShield Does

PsyberShield helps you:

- scan a target URL
- crawl in-scope pages
- discover a local app target on a VPS when no URL is supplied
- inspect local environment and server layout
- generate safe fix artifacts
- apply one real local fix lane with backup and validation
- save reports, baselines, audit history, and comparison summaries

PsyberShield is intentionally defensive:

- it warns before risky commands
- it never prints secret values
- it prefers read-only behavior unless you explicitly ask for fixes
- it keeps a clear trail in the terminal and in saved outputs

Command Map

Command	What it does	Typical use
‘scan‘	Scans one target page or a discovered local target; supports browser-assisted logir
‘crawl‘	Crawls in-scope links across multiple pages; supports browser-assisted login and ‘
‘report‘	Re-renders or previews a saved report	Review an existing JSON, Markdown, or HTM
‘audit‘	Shows the append-only audit trail	Review scans and fix events
‘baseline‘	Saves a scan snapshot for later comparison	Track a known-good state

| ‘compare‘ | Compares two saved scans and crawl coverage | See what changed |
| ‘drift‘ | Compares saved reports and highlights baseline drift | Watch for scan, file, log,
| ‘secrets‘ | Scans files for obvious secret exposure | Check logs and configs for leaked valu
| ‘vuln scan‘ | Inventories local software versions and matches a small bundled offline adviso
| ‘bundle‘ | Packages a report and related artifacts into a ZIP archive | Share a handoff bund
| ‘doctor‘ | Checks the local machine and app environment | Health check without a URL |
| ‘server-check‘ | Discovers the server layout and scans the local app target | VPS/server dis
| ‘incident‘ | Detects suspicious activity in logs and can apply a denylist or fail2ban snippe
| ‘watch‘ | Monitors logs, file drift, and process activity in a snapshot or follow loop; writ
| ‘timeline‘ | Shows the chronological order of findings and containment actions | Saved incid
| ‘integrity‘ | Compares monitored files against a saved baseline | File integrity drift monit
| ‘fix --local‘ | Applies the first real local edit lane | Backup, edit, validate, rollback if
| ‘demo‘ | Runs the local demo site | Test target for development |
| ‘demo-site‘ | Compatibility alias for the local demo site | Legacy command support |

Quick Start

Scan a target:

```
.\venv\Scripts\python.exe -m app.main scan https://example.com
```

Discover a VPS target without typing a URL:

```
.\venv\Scripts\python.exe -m app.main scan
```

Crawl more than one page:

```
.\venv\Scripts\python.exe -m app.main crawl https://example.com --max-pages 20 --max-depth 2
.\venv\Scripts\python.exe -m app.main crawl https://example.com --profile safe-vps
```

Generate an HTML report with a default filename under ‘outputs/‘:

```
.\venv\Scripts\python.exe -m app.main crawl https://example.com --html-output
```

Run the server discovery workflow:

```
.\venv\Scripts\python.exe -m app.main server-check --yes
```

Check logs for suspicious activity and optionally apply containment:

```
.\venv\Scripts\python.exe -m app.main incident --logs outputs\access.log --apply-blocks
.\venv\Scripts\python.exe -m app.main integrity . --baseline baselines\integrity.json
```

```
.\venv\Scripts\python.exe -m app.main timeline outputs\incident.json --audit-log outputs\audit
.\venv\Scripts\python.exe -m app.main watch --logs outputs\access.log --json-output
.\venv\Scripts\python.exe -m app.main watch --follow --interval 30 --tail-file outputs\access.
.\venv\Scripts\python.exe -m app.main vuln scan --html-output
```

Capture fresh snapshots from live sources:

```
.\venv\Scripts\python.exe -m app.main incident --live --tail-file outputs\access.log
.\venv\Scripts\python.exe -m app.main incident --live --event-log-name System --fail2ban-outpu
```

Write a fail2ban-style snippet:

```
.\venv\Scripts\python.exe -m app.main incident --logs outputs\access.log --fail2ban-output out
```

View a saved incident timeline:

```
.\venv\Scripts\python.exe -m app.main timeline outputs\incident.json --audit-log outputs\audit
```

Generate rate-limit and maintenance-mode presets:

```
.\venv\Scripts\python.exe -m app.main incident --logs outputs\access.log --rate-limit-output o
```

Compare saved reports for drift:

```
.\venv\Scripts\python.exe -m app.main drift baselines\scan.json outputs\scan.json --json-outpu
```

Scan for obvious secret exposure:

```
.\venv\Scripts\python.exe -m app.main secrets . --markdown-output outputs\secrets.md
```

Inventory local software versions and match bundled offline advisories:

```
.\venv\Scripts\python.exe -m app.main vuln scan
.\venv\Scripts\python.exe -m app.main vuln scan --json-output outputs\vuln.json --html-output
.\venv\Scripts\python.exe -m app.main vuln scan --inventory-only
```

Bundle a report and related artifacts:

```
.\venv\Scripts\python.exe -m app.main bundle outputs\incident.json --artifact outputs\incident
```

Apply the first real local fix lane:

```
.\venv\Scripts\python.exe -m app.main fix --local --yes
```

Safety and Permission Prompt

Risky commands ask for confirmation by default:

- ‘scan‘
- ‘crawl‘
- ‘server-check‘
- ‘incident‘
- ‘fix --local‘

The prompt is a reminder that you should only test systems you own or have explicit permission to test.

Use ‘--yes‘ to skip the prompt in trusted automation.

Read-only commands stay quiet:

- ‘doctor‘
- ‘report‘
- ‘audit‘
- ‘compare‘
- ‘baseline‘

Target Resolution

‘scan‘ and ‘crawl‘ accept a target URL directly.

If you omit the URL, PsyberShield resolves a target in this order:

1. explicit URL on the command line
2. ‘--env-file‘
3. the apps own ‘.env‘
4. OS environment variables
5. server discovery on a VPS

The common environment variables are:

- ‘APP_URL‘
- ‘TARGET_URL‘
- ‘BASE_URL‘

When PsyberShield discovers a local app target, it prints a short ‘Discovery:‘ line and then a fuller application context block.

Output Files and Paths

PsyberShield writes user-facing artifacts into ‘outputs/‘.

Supported output flags:

- ‘--json-output‘
- ‘--markdown-output‘
- ‘--html-output‘
- ‘--output‘
- ‘--audit-log‘
- ‘--log-file‘

You can use them in three ways:

- pass a full path
- pass a relative path
- pass the flag without a path and let PsyberShield create a timestamped filename under ‘outputs/‘

Examples:

```
.\venv\Scripts\python.exe -m app.main scan https://example.com --html-output
.\venv\Scripts\python.exe -m app.main crawl https://example.com --json-output outputs\crawl.js
.\venv\Scripts\python.exe -m app.main baseline https://example.com --output
```

If you use a Windows-rooted path like ‘\outputs\crawl.html‘, PsyberShield treats it as project-relative and prints the resolved path.

‘scan‘

‘scan‘ checks a single target entrypoint.

Syntax:

```
.\venv\Scripts\python.exe -m app.main scan [URL]
```

Important flags:

- ‘--env-file PATH‘ reads a specific ‘.env‘
- ‘--timeout SECONDS‘ changes the request timeout
- ‘--profile quick‘, ‘--profile full‘, and ‘--profile safe-vps‘ tune the scan defaults
- ‘--policy PATH‘ loads a policy file
- ‘--yes‘ skips the permission prompt
- ‘--preview-fixes‘ shows proposed fixes only
- ‘--interactive‘ lets you choose generate artifacts or local fixes
- ‘--generate-fixes‘ generates safe local remediation artifacts
- ‘--apply-fixes‘ is a legacy alias for ‘--generate-fixes‘
- ‘--login-url‘, ‘--auth-method‘, ‘--username‘, ‘--password‘, ‘--password-env‘, ‘--cookie‘, ‘--session-file‘, ‘--storage-state‘, and browser auth flags enable authenticated scanning
- ‘--json-output‘, ‘--markdown-output‘, ‘--html-output‘ save reports

What to expect:

- one target page
- headers, cookies, TLS summary, exposed files, WAF signals
- suggested fix plans when report-only findings exist
- each fix plan gets a confidence label: report only, generate artifact, safe local fix, or needs manual approval
- optional saved reports in JSON, Markdown, and HTML

Example:

```
.\venv\Scripts\python.exe -m app.main scan https://example.com --json-output --markdown-output
.\venv\Scripts\python.exe -m app.main scan https://example.com --profile quick
```

‘crawl‘

‘crawl‘ starts from one page and follows in-scope links across multiple pages.

Syntax:

```
.\venv\Scripts\python.exe -m app.main crawl [URL]
```

Important flags:

- ‘--max-pages N‘
- ‘--max-depth N‘
- ‘--include REGEX‘

- ‘--exclude REGEX‘
- ‘--same-host-only‘ and ‘--allow-offsite‘
- ‘--seed-robots‘ and ‘--seed-sitemap‘
- ‘--env-file PATH‘
- ‘--yes‘
- authentication and browser-auth flags from ‘scan‘
- report output flags

What to expect:

- a crawl summary with pages visited, unique URLs, and seed sources
- findings grouped by page in the terminal and saved reports
- repeated findings collapsed into an ‘Affected URLs‘ list
- optional seeded discovery from ‘robots.txt‘ and ‘sitemap.xml‘

Example:

```
.\venv\Scripts\python.exe -m app.main crawl https://example.com --max-pages 20 --max-depth 2 -
```

Authenticated Scanning and Crawling

PsyberShield supports generic authenticated flows for protected content.

Supported auth modes:

- HTTP JSON login
- HTTP form login
- raw cookie reuse
- saved session files
- browser storage-state reuse
- browser-assisted login for JS-heavy flows

Common flags:

- ‘--login-url‘
- ‘--auth-method json|form|browser‘
- ‘--username‘
- ‘--password‘
- ‘--password-env‘
- ‘--user-field‘
- ‘--pass-field‘
- ‘--cookie‘

- ‘--session-file‘
- ‘--save-session‘
- ‘--storage-state‘
- ‘--save-storage-state‘
- ‘--browser-username-selector‘
- ‘--browser-password-selector‘
- ‘--browser-submit-selector‘
- ‘--browser-headless‘ and ‘--browser-headed‘
- ‘--auth-check-url‘

Recommended pattern:

```
.\venv\Scripts\python.exe -m app.main crawl ‘
  https://example.com ‘
  --login-url /auth/login ‘
  --auth-method json ‘
  --username alice ‘
  --password-env PsyberShield_PASSWORD ‘
  --auth-check-url /account
```

Browser-assisted quick example:

Browser auth requires either ‘--login-url‘ or ‘--storage-state‘ so PsyberShield can establish the session before it scans.

```
.\venv\Scripts\python.exe -m app.main scan ‘
  https://example.com ‘
  --auth-method browser ‘
  --login-url /auth/login ‘
  --browser-username-selector ’input[name="identifier"]’ ‘
  --browser-password-selector ’input[name="password"]’ ‘
  --username alice ‘
  --password-env PsyberShield_PASSWORD ‘
  --auth-check-url /account
```

Storage-state reuse example:

```
.\venv\Scripts\python.exe -m app.main crawl ‘
  https://example.com ‘
  --storage-state browser\storage_state.json ‘
  --auth-check-url /account
```

What to expect:

- PsyberShield logs in before crawling or scanning protected pages
- it confirms that login worked by checking a protected URL when you provide
  ‘--auth-check-url‘
- it reuses the same session for the scan or crawl

‘report‘

‘report‘ re-renders or previews a saved scan report from disk.

Syntax:

```
.\venv\Scripts\python.exe -m app.main report <REPORT_FILE>
```

Accepted inputs:

- ‘.json‘
- ‘.md‘
- ‘.html‘
- ‘.htm‘

Optional output flags:

- ‘--json-output‘
- ‘--markdown-output‘
- ‘--html-output‘

What to expect:

- JSON reports are rendered back into the normal terminal report view
- Markdown and HTML files can be copied to new destinations
- bare output flags create files under ‘outputs/‘

‘audit‘

‘audit‘ shows the append-only audit history.

Important flags:

- ‘--audit-log‘ and legacy alias ‘--log-file‘
- ‘--last N‘

- ‘--event NAME‘
- ‘--target TEXT‘
- ‘--json-output‘

What to expect:

- filtered audit events in the terminal
- optional JSON export of the filtered audit list
- audit entries for scans, baselines, and fix actions

Example:

```
.\venv\Scripts\python.exe -m app.main audit --last 25 --json-output
```

‘baseline‘

‘baseline‘ saves a scan snapshot for later comparison.

Important flags:

- ‘--label NAME‘
- ‘--output‘
- ‘--audit-log‘
- ‘--timeout‘
- ‘--policy‘

What to expect:

- a baseline JSON snapshot
- a companion ‘.meta.json‘ metadata file
- discovery details saved alongside the snapshot

Example:

```
.\venv\Scripts\python.exe -m app.main baseline https://example.com --label vps-west --output
```

‘compare‘

‘compare‘ compares two saved scan files.

Important flags:

- ‘--markdown-output‘
- ‘--html-output‘

What to expect:

- fixed findings
- new findings
- unchanged findings
- crawl coverage deltas when the inputs come from ‘crawl‘
- a short terminal note when crawl coverage changes

Example:

```
.\venv\Scripts\python.exe -m app.main compare old.json new.json --html-output
```

‘doctor‘

‘doctor‘ checks the local machine and app environment, including suspicious listeners and outbound connections. It also adds a deployment profile hint so you can tell whether the host looks like a local development box, a service-backed server, or a likely VPS-style deployment. Its readiness score is paired with a short breakdown of the checks that most affected it, and the report labels the overall state as ‘ready‘, ‘warning‘, or ‘danger‘.

‘doctor‘ also accepts ‘--json-output‘, ‘--markdown-output‘, and ‘--html-output‘ if you want a saved report file.

What it checks:

- OS info
- Python version
- ‘.env‘ presence
- output folder writability
- app config paths
- common Nginx config paths
- localhost ports
- safe secret-status signals without printing secret values

Example:

```
.\venv\Scripts\python.exe -m app.main doctor --env-file C:\path\to\autoentrytrack\.env
```

‘server-check‘

‘server-check‘ is the server-facing discovery flow.

What it does:

- checks Nginx and systemd hints
- discovers the app target
- resolves the apps own ‘.env‘ when possible
- scans the discovered local target
- surfaces the deployment profile hint and the overall ‘ready‘ / ‘warning‘ / ‘danger‘ state

Important flags:

- ‘--env-file‘
- ‘--nginx-config‘
- ‘--timeout‘
- ‘--yes‘

Example:

```
.\venv\Scripts\python.exe -m app.main server-check --yes
```

‘fix --local‘

‘fix --local‘ is the first real live-edit lane.

What it does:

1. discovers a supported local file
2. backs it up
3. applies the change
4. validates it
5. rolls back on failure

What to expect:

- the target file is backed up before any edit
- the terminal shows the backup path and validation command
- the chosen local fix must be supported by the current lane
- unsupported findings stay in generate-artifact mode

Example:

```
.\venv\Scripts\python.exe -m app.main fix --local --yes
```

‘--generate-fixes‘ and the legacy ‘--apply-fixes‘

‘--generate-fixes‘ creates local remediation artifacts and notes.

‘--apply-fixes‘ still works as a backwards-compatible alias, but ‘--generate-fixes‘ is the current name.

What to expect:

- a backup of the generated artifact, if one already exists
- a file under ‘outputs/generated/‘
- a matching note and audit entry

‘demo‘

‘demo‘ runs a local test target for development.

Example:

```
.\venv\Scripts\python.exe -m app.main demo --port 8000
```

‘demo-site‘ remains available as a compatibility alias.

‘.env‘ Variables

These are the main environment variables PsyberShield reads:

Variable	Used by	Purpose
‘APP_URL‘	‘scan‘	Primary fallback target
‘TARGET_URL‘	‘scan‘	Secondary fallback target
‘BASE_URL‘	‘scan‘	Final fallback target
‘DEBUG‘	‘doctor‘, ‘server-check‘	Warns about noisy debug mode
‘SECRET_KEY‘	‘doctor‘, ‘server-check‘	Presence/weakness check only
‘SERVER_NAME‘	‘doctor‘, ‘server-check‘	Reported as present or missing
‘DATABASE_URL‘	‘doctor‘, ‘server-check‘	Reported as present or missing
‘SMTP_PASSWORD‘	‘doctor‘, ‘server-check‘	Reported as present or missing

| ‘PsyberShield_PASSWORD‘ | ‘scan‘, ‘crawl‘ | Secret value read by ‘--password-env‘ from the s

Saved Outputs

Current output folders:

- ‘outputs/‘ for user-facing artifacts
- ‘outputs/generated/‘ for generated fix artifacts
- ‘outputs/backups/‘ for backup files
- ‘outputs/remediation/‘ for remediation notes

Useful output examples:

```
.\venv\Scripts\python.exe -m app.main scan https://example.com --html-output
.\venv\Scripts\python.exe -m app.main crawl https://example.com --markdown-output outputs\craw
.\venv\Scripts\python.exe -m app.main audit --json-output
```

What the Output Means

Typical scan output includes:

- target summary
- finding counts
- severity counts
- top categories
- WAF or CDN signals
- TLS summary when available
- a findings table
- proposed fixes for report-only issues

Typical crawl output adds:

- pages visited
- unique URLs
- seed sources
- per-page findings
- grouped repeated findings with affected URL lists

Typical doctor or server-check output adds:

- root path
- OS

- Python version
- environment file discovery
- Nginx/systemd hints
- suspicious listener and outbound connection activity
- localhost port checks

Typical incident output adds:

- source log paths
- suspect IPs and blocked IPs
- log family hints such as ‘apache-access‘, ‘apache-error‘, ‘auth‘, ‘auth-middleware‘, ‘gunicorn‘, ‘ssh‘, ‘sudo‘, ‘systemd‘, and ‘uwsgi‘
- optional denylist, fail2ban, rate-limit, or maintenance-mode containment artifacts

Typical timeline output adds:

- ordered finding and containment events
- timestamps when the log lines or audit entries provided them
- source paths and audit log references for each event

Limitations and Boundaries

PsyberShield is intentionally defensive and best effort:

- it is not a substitute for a full SIEM, IDS, or dedicated EDR platform
- browser auth depends on the page structure, selectors, and whether the app accepts the login flow you provide
- some findings are heuristic and may need manual review before you treat them as confirmed issues
- WAF, TLS, and process/port signals may be unavailable or incomplete depending on the target and local environment
- saved reports and bundles only include what you asked PsyberShield to capture
- only the supported local fix lane is applied automatically; other findings stay in report or artifact form unless you act on them yourself
- you should only scan systems you own or have explicit permission to test

Integrity

‘integrity‘ compares monitored files in a root directory against an optional saved baseline.

Example:

```
.\venv\Scripts\python.exe -m app.main integrity . --baseline baselines\integrity.json --json-o

Typical integrity output adds:

- the monitored root
- the baseline path when one is supplied
- changed, missing, and new monitored files
- the file category, kind, hash, size, and timestamp for each tracked file
```

Baseline Workflow

```
Create a baseline after a known-good deployment:

.\venv\Scripts\python.exe -m app.main integrity . --create-baseline baselines\integrity.json

Refresh the baseline after approved changes only:

.\venv\Scripts\python.exe -m app.main integrity . --create-baseline baselines\integrity.json

Use the baseline in ‘watch‘ when you want live drift checks:

.\venv\Scripts\python.exe -m app.main watch --baseline baselines\integrity.json --logs outputs

Operational notes:

- keep the baseline file in a team-owned location
- refresh it only after approved changes land
- expect ordinary drift from releases, config edits, and content updates
- record what changed when you refresh it so the baseline stays trusted
```

Drift

```
‘drift‘ compares two saved reports of the same type and surfaces changes that matter
for defensive operations.

Example:

.\venv\Scripts\python.exe -m app.main drift baselines\scan.json outputs\scan.json --html-outpu

Typical drift output adds:
```

- the report type being compared
- baseline and current report paths
- changed findings or checks
- file, log, header, or config drift summaries depending on the report type

Secret Exposure

‘secrets‘ scans likely config and log files for obvious secret exposure and redacts the evidence before it is stored in the report.

Example:

```
.\venv\Scripts\python.exe -m app.main secrets . --json-output outputs\secrets.json
```

If you do not pass any output flags, ‘secrets‘ writes JSON, Markdown, and HTML reports under ‘outputs/‘ automatically.

Typical secret-exposure output adds:

- the monitored root
- candidate source file count
- redacted snippets from matching lines
- a short recommendation for each exposure

Vulnerability Inventory

‘vuln scan‘ inventories local software versions, parses Python dependency manifests, and compares discovered exact versions with advisory sources.

Example:

```
.\venv\Scripts\python.exe -m app.main vuln scan
.\venv\Scripts\python.exe -m app.main vuln scan --json-output outputs\vuln.json --html-output
.\venv\Scripts\python.exe -m app.main vuln scan --inventory-only
.\venv\Scripts\python.exe -m app.main vuln scan --osv --osv-cache outputs\advisory-cache\osv
```

This version checks common local commands such as Nginx, Apache, OpenSSL, Python, Node, npm, and PHP, then records the versions it can prove. It also parses Python dependencies from ‘requirements.txt‘ and ‘pyproject.toml‘, including exact pins, version ranges, extras, environment markers, and direct references.

Default behavior stays offline and uses bundled rules for a few well-known advisories,

including Apache HTTP Server 2.4.49/2.4.50 and OpenSSL 3.0.0 through 3.0.6. Use ‘--osv‘ to explicitly query OSV for parsed Python dependency manifests with exact versions, such as ‘flask==3.0.0‘. Non-exact constraints such as ‘django>=5.0‘ stay in the inventory but are not treated as confirmed installed versions. OSV responses are cached under ‘outputs\advisory-cache\osv‘ by default, or under the path supplied with ‘--osv-cache‘.

Important limitation: this is still not a full NVD, distro-advisory, or vendor-backport scanner. Treat dependency matches as strong signals, and confirm system package findings against the operating-system vendor before making production decisions.

Typical vulnerability-inventory output adds:

- the monitored root
- the software components checked
- versions discovered from local command output
- Python dependencies discovered from dependency manifests
- local advisory matches when the bundled ruleset applies
- OSV dependency matches when ‘--osv‘ is enabled
- source and confidence labels for each advisory finding
- a note that distro backports and vendor advisories should be confirmed

Automatic App Security Checks

PsyberShield can run automatically when you schedule its commands with the operating system. For the next product phase, keep the schedule focused on the hosted web app first: scan the app, crawl the app, compare saved reports, and collect dependency advisory reports. Server/VPS hardening checks can run on a slower schedule until the server-side detection work is expanded further.

Recommended app-first command set:

```
pshield scan https://example.com --profile quick --json-output outputs\auto-scan.json --markdo
pshield crawl https://example.com --profile full --seed-robots --seed-sitemap --json-output ou
pshield vuln scan --osv --json-output outputs\vuln-app.json --markdown-output outputs\vuln-app
pshield secrets . --json-output outputs\secrets.json --markdown-output outputs\secrets.md --ht
```

For authenticated apps, reuse the same browser/session flags that work manually:

```
pshield crawl https://example.com --auth-method browser --login-url /auth/login --browser-user
pshield crawl https://example.com --storage-state browser\storage-state.json --auth-check-url
```

What to expect:

- ‘scan‘ gives a quick page-level app security snapshot
- ‘crawl‘ discovers and checks multiple in-scope pages
- ‘vuln scan --osv‘ reports exact Python dependency advisories when manifests are available
- ‘secrets‘ flags obvious secret exposure in local project files and reports redacted evidence
- saved reports land wherever the output flags point, usually under ‘outputs/‘
- scheduled runs should write timestamped filenames if you want historical reports instead of overwriting the same file

Linux cron example:

```
0 2 * * * cd /opt/psybershield && /opt/psybershield/venv/bin/pshield crawl https://example.com
15 2 * * * cd /opt/psybershield && /opt/psybershield/venv/bin/pshield vuln scan --osv --html-o
```

Linux systemd timer pattern:

```
# /etc/systemd/system/psybershield-app-scan.service
[Unit]
Description=PsyberShield scheduled app scan

[Service]
Type=oneshot
WorkingDirectory=/opt/psybershield
ExecStart=/opt/psybershield/venv/bin/pshield crawl https://example.com --profile full --seed-r

# /etc/systemd/system/psybershield-app-scan.timer
[Unit]
Description=Run PsyberShield app scan nightly

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true

[Install]
WantedBy=timers.target
```

Windows Task Scheduler example:

```
$action = New-ScheduledTaskAction -Execute "C:\Users\Bernard\Desktop\learning\Turan\venv\Scrip
$trigger = New-ScheduledTaskTrigger -Daily -At 2am
Register-ScheduledTask -TaskName "PsyberShield App Crawl" -Action $action -Trigger $trigger -D
```

Useful follow-up workflow:

```
pshield compare baselines\crawl.json outputs\auto-crawl.json --html-output outputs\crawl-compa
pshield bundle outputs\auto-crawl.json --artifact outputs\crawl-compare.html --bundle-output c
```

Safety and limitations:

- schedule scans only for apps you own or have explicit permission to test
- keep crawls passive and scoped unless you intentionally widen scope
- authenticated schedules should use environment variables or ‘.env‘ values, not passwords in task definitions
- app detection focuses on web security posture, crawl coverage, headers, cookies, exposed files, and dependency advisories
- deeper server/VPS detection, patch validation, and automatic containment should be phased in later with explicit policy and audit controls

Private Web Dashboard

PsyberShield can run as a private FastAPI dashboard on your VPS. This is the preferred source-protection model for a small team: the Python source stays on your server, and users access HTML pages through a private URL, VPN, or localhost tunnel.

Install the web dependencies:

```
.\venv\Scripts\python.exe -m pip install -e .
```

Set the dashboard environment variables:

```
$env:PSHIELD_DATABASE_URL="postgresql+psycopg://psybershield:change-me@127.0.0.1:5432/psybersh
$env:PSHIELD_SECRET_KEY="replace-with-a-long-random-dashboard-secret"
$env:PSHIELD_OUTPUT_DIR="outputs/web"
$env:PSHIELD_WEB_HOST="127.0.0.1"
$env:PSHIELD_WEB_PORT="8787"
$env:PSHIELD_ADMIN_EMAIL="admin@example.com"
$env:PSHIELD_ADMIN_PASSWORD="replace-with-a-temporary-admin-password"
```

Start the web dashboard:

```
pshield web --host 127.0.0.1 --port 8787

Start the background worker in a second terminal or service:

pshield worker

For Postgres deployments, run the schema migration before production use:

alembic upgrade head

V1 dashboard capabilities:

- local email/password login
- roles: ‘admin‘, ‘operator‘, and ‘viewer‘
- admin-created users
- target management
- queued jobs for ‘scan‘, ‘crawl‘, ‘vuln scan‘, ‘secrets‘, ‘baseline‘, ‘compare‘, and
  ‘bundle‘
- job ownership so the UI can show My Scans, Team Scans, and Recent Scans over time
- report metadata stored in Postgres
- JSON, Markdown, and HTML report downloads from ‘PSHIELD_OUTPUT_DIR‘
- JSON, Markdown, and HTML report previews inside the dashboard
- auto-refreshing queued/running job detail pages
- CSRF protection for authenticated form submissions
- audit events linked to jobs and users

V1 intentionally does not expose:

- ‘fix --local‘
- live containment
- process killing
- account disabling
- file quarantine
- firewall or Nginx changes

Linux systemd service example for the dashboard:

[Unit]
Description=PsyberShield Web Dashboard
After=network.target postgresql.service

[Service]
```

```
WorkingDirectory=/opt/psybershield
EnvironmentFile=/opt/psybershield/.env
ExecStart=/opt/psybershield/venv/bin/pshield web --host 127.0.0.1 --port 8787
Restart=always

[Install]
WantedBy=multi-user.target
```

The repository also includes ready-to-adapt examples:

- ‘deploy/systemd/psybershield-web.service‘
- ‘deploy/systemd/psybershield-worker.service‘
- ‘deploy/nginx/psybershield.conf‘

Linux systemd service example for the worker:

```
[Unit]
Description=PsyberShield Web Worker
After=network.target postgresql.service

[Service]
WorkingDirectory=/opt/psybershield
EnvironmentFile=/opt/psybershield/.env
ExecStart=/opt/psybershield/venv/bin/pshield worker
Restart=always

[Install]
WantedBy=multi-user.target
```

Recommended exposure:

- bind the app to ‘127.0.0.1‘
- access it through SSH tunnel, Tailscale, WireGuard, or a private reverse proxy
- avoid public internet exposure until rate limits, CSRF hardening, and production auth review are complete
- back up Postgres and ‘PSHIELD_OUTPUT_DIR‘

Containment Planning

PsyberShield separates containment planning from live containment actions. The current foundation defines recommendation, action, and result models so future defensive features can be explicit, reversible where possible, and audited.

Current status:

- high-risk watch findings can be mapped into containment recommendations
- recommendation types include IP blocking, rate limiting, maintenance mode, file quarantine, process termination, account disablement, and manual review
- generated actions default to dry-run planning and require approval
- no automatic process killing, account disabling, file quarantine, or firewall changes are executed by these models

Bundle

‘bundle‘ packages a report together with related artifacts into a ZIP archive.

Example:

```
.\venv\Scripts\python.exe -m app.main bundle outputs\incident.json --artifact outputs\incident
```

If you omit ‘--bundle-output‘, PsyberShield creates a report-named archive like ‘incident.bundle.zip‘ and writes a matching manifest inside the ZIP.

Typical bundle output adds:

- the archive path
- the source report path
- the files included in the archive
- a small manifest inside the ZIP

Notifications

‘incident‘, ‘integrity‘, and ‘timeline‘ can send a short report summary after they finish.

Common flags:

- ‘--webhook-url‘
- ‘--slack-webhook-url‘
- ‘--discord-webhook-url‘
- ‘--email-to‘
- ‘--email-from‘
- ‘--smtp-host‘
- ‘--smtp-port‘

- ‘--smtp-username‘
- ‘--smtp-password-env‘
- ‘--smtp-starttls‘ and ‘--no-smtp-starttls‘

Example:

```
.\venv\Scripts\python.exe -m app.main incident --logs outputs\access.log --webhook-url https:/
.\venv\Scripts\python.exe -m app.main integrity . --baseline baselines\integrity.json --email-
.\venv\Scripts\python.exe -m app.main timeline outputs\incident.json --audit-log outputs\audit
```

Troubleshooting

If a bare output flag creates a file and you cannot find it:

- PsyberShield writes under ‘outputs/‘ in the current project directory
- if you pass ‘\outputs\file.html‘, PsyberShield treats it as project-relative and tells you the resolved path
- if you pass a full absolute path, PsyberShield respects it

If ‘crawl‘ only reaches login or redirect pages:

- add auth flags
- add ‘--auth-check-url‘
- confirm that the login worked before you trust the crawl results

If ‘server-check‘ finds a local app target but still points to the repo ‘.env‘:

- make sure the discovered ‘EnvironmentFile‘ or working directory is correct
- pass ‘--env-file‘ if you want to override discovery

Maintaining This Guide

This guide is meant to be updated as PsyberShield grows.

When you add a new command or flag:

1. update the CLI help in ‘app/main.py‘
2. update this guide
3. update the README quick-start sections if needed
4. regenerate the PDF